

Artificial Intelligence (CSE422)

Introduction (L-2)

What is AI?

Machine learning is a subfield of AI.

From the two dimensions - human vs. rational and thought vs. behavior - there are four possible combinations.

Thinking Humane | # Thinking Rationally

Acting Humane | # Acting Rationally

Acting humanly

Programming a computer to pass a rigorously applied test provides plenty to work on. The computer would need the following capabilities:

- natural language processing to communicate successfully in a human language;
- knowledge representation to store what it knows or hears;
- automate reasoning to answer questions and to draw new conclusion;
- machine learning to adapt new circumstances and to detect and extrapolate patterns.

Thinking humanly

To say that a program thinks like a human, we must know how humans think. We can learn about human thought in three ways:

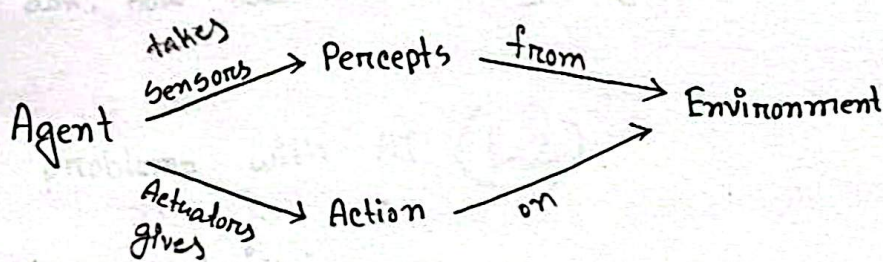
- introspection - trying to catch our own thought as they go by;
- psychological experiments - observing a person in action;
- brain imaging - observing the brain in action.

Thinking rationally

Acting Rationally

→ An agent is something that acts. All computer programs do something, but computer agents are expected to do more: operate autonomously, perceive their environment, persist over a prolonged time period, adapt to change, and create and pursue goals.

→ A rational agent is one that acts so as to achieve the best outcome or, when there is uncertainty, the best expected outcome.



→ Autonomous agents are advanced AI systems that can perform complex tasks and make decisions independently, without constant human intervention.

Approach - Thinking human

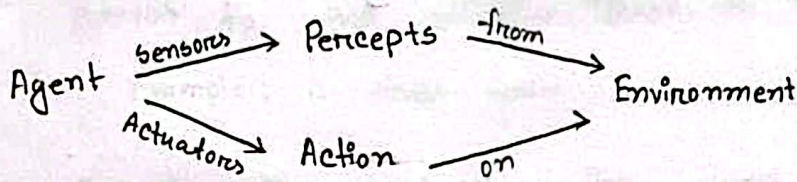
Acting human

Thinking Rationally

Acting Rationally

1) Ability to gain percepts

2) Ability to learn



Agent: takes percepts from environment using sensors.

Agent gives actions on environment using actuators.

Always ask: How does it help my agent become more intelligent?

Solving problems with AI (L2)

Searching Strategies:

- Informed Search: Informed search strategies, such as A* and Best-first Search, utilize heuristics to efficiently find solutions, focusing the search towards more promising paths. These strategies are essential in scenarios like real-time pathfinding for autonomous vehicles.

- Local Search: Local search methods are crucial for tackling optimization problems where finding an optimal solution might be too time-consuming. These methods include: simulated Annealing and Hill climbing, are invaluable for tasks such as resource allocation and scheduling where a near-optimal solution is often sufficient.

- Adversarial search: Adversarial search techniques are essential for environments where agents compete against each other, such as in board games or market competitions. Understanding strategies like Minimax, and Alpha-Beta Pruning allows one to predict and counter opponent's move effectively.

Some keywords:

* Agent: In AI, an agent is an entity that perceives its environment through sensors and acts upon that environment using actuators.

* Rational Agent: A rational agent acts to achieve the best outcome or, when there is uncertainty, the best expected outcome.

* Autonomous Agent: An autonomous agent is a type of rational agent that can learn from its own experiences and actions. It can adjust its behavior based on new information, making up for any initial gaps or inaccuracies in its knowledge.

* Task Environment: In AI, the environment refers to everything external to the agent that it interacts with or perceives to make decisions and achieve its goals.

Properties of Task Environment

Fully observable vs. Partially observable

- Fully observable: In these environments, the agent's sensors provide access to the complete state of environment at all times.

Example: a chess game

- Partially observable: Here, the agent only has partial information about the environment due to limitations in sensor capabilities or because some information is inherently hidden.

Example: a poker game

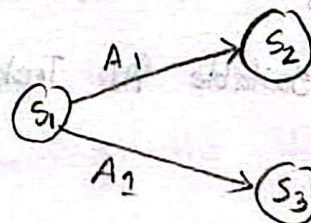
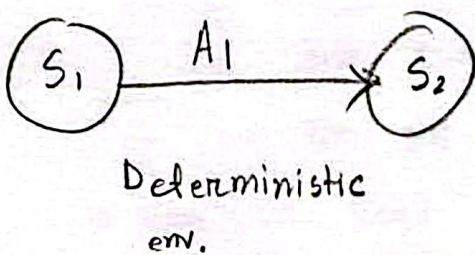
Deterministic vs. Stochastic

- Episodic vs. Sequential → current choice affects all future decisions.

- Static vs. Dynamic
→ env. doesn't change

- Discrete vs. Continuous
→ possible states or actions is infinite

- Competitive vs. Cooperative
→ Agents work together



(Agent don't know where to go)

Types of agents

- Simple Reflex Agents $\rightarrow$ if a condition is met, an action is taken
- Model - Based Reflex Agents $\rightarrow$ uses a model of the world to decide their action
- Goal-Based Agents $\rightarrow$ act to achieve goals
- Utility-Based Agents $\rightarrow$ They choose actions based on which outcome provides the greatest benefit according to this utility

Steps of Problem Formulation

1. Define the goal
2. Identify the initial State
3. Determine the possible Actions
4. Define the Transition Model
5. Establish the Goal Test
6. Define the Path Cost
7. Consider Any Constraints
8. Select the Suitable AI Technique

Examples of problem formulation

1 City Traffic Navigation (Solved by Informed Search)

• Problem Formulation:

- Goal: To find the quickest route from a standing point (origin) to a destination (end point) while considering current traffic conditions.

- States: Each state represents a geographic location within the city's road network.

- Initial State: The specific standing location of the vehicle.

- Actions: From any given state (location), the actions available are the set of all possible roads that can be taken next.

- Transition Model: Moving from one location to another via a chosen road or intersection.

- Goal Test: Determines whether the current location is the destination.

- Path Cost: Each step cost can be a function of travel time, which depends on factors such as distance and current traffic. The total path is the sum of the step costs, representing the total travel time.

• Heuristic Used:

- Time Estimation:

- Distance:

Imp: Will be studying Deterministic Environment

* BFS $\rightarrow$ check the designed state is reachable or not.
 $\downarrow$ if yes

Sequence of actions needed to be taken.

= # 4 criteria based on which we judge if an agent is good or not:
 * The number of state/node needed to be generated/visited $\leftarrow$ We use this as Time Comp.
 * instead of vertices or edges, we use terms like:

1) Space Complexity

2) Time Complexity

3) Completeness

4) Optimality

- branching factor (b)
 $\hookrightarrow$ max no. branch state can have
- max depth, (m)
 $\hookrightarrow$ total depth of tree
- Solution depth (d)

* A complete algo will return a solution if the problem is solvable.
 Doesn't matter if it's good or bad or mid solution.

$\hookrightarrow$ the depth at which you find the nearest/shallowest solution

* Optimality $\rightarrow$ the least cost solution.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete ?	Yes	Yes	No	No	Yes
Time	$O(b^m)$		$O(b^d)$		
Space			$O(b \times m)$		
Optimal ?	Yes	Yes	No	No	Yes

BFS complete.

DFS not compel.

Informed Search Algorithm (L3)

Heuristic Function

Dijkstra == best first search

In the context of informed search algorithms, a heuristic is a technique that helps the algorithm estimate the cost (often the shortest path or least costly path) from current state to the goal state.

Heuristic
→ An estimation of the cheapest path cost between any node N and the Goal.

Key Characteristics of heuristics in Search Algorithm

Estimation: A heuristic function estimates the cost to reach the goal from a current node. This estimation does not need to be exact but should never overestimate.

Greedy Best-First Search

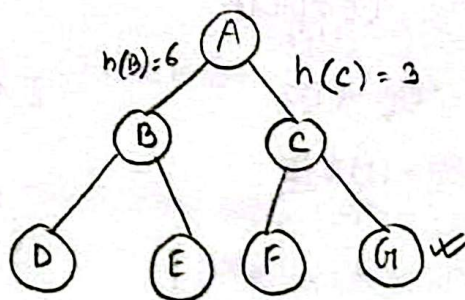
→ evaluation function, $f(n) = h(n)$

→ Time complexity = $O(|V|)$

→ With a good heuristic function, however, the complexity can be reduced substantially, on certain problems reaching $O(bm)$.

→ Evaluation function $f(n)$ evaluates the utility of a state.

Example: $h(A) = 5$ means an estimation that cheapest cost from node A to goal node is 5.



Condition on heuristic for A^* to be optimal:

Admissibility

An admissible heuristic never overestimates the cost to reach the goal. This makes it optimistic about the path costs.

* If $h(H)$ not 98, $h(H) = 138$
the $H \rightarrow I$ cost 101 < 138

this will make the path not optimal.

optimal?

↳ h admissible
↳ optimal
↳ h not admissible
↳ not optimal

$\therefore h(n) < n \rightarrow$ goal last value (addition)

Example:

$\frac{E}{\text{L}}$ → choose the smaller one
↳ actual cost = $80 + 97 + 101 = 278$

$h(E) = 253 \quad \therefore h(E) < 278$

$\therefore$ admissible

→ If the condition breaks for even a single node,
then the graph is not admissible.

* Tree search will be admissible if $h(n)$ never overestimates the actual cheapest path cost.

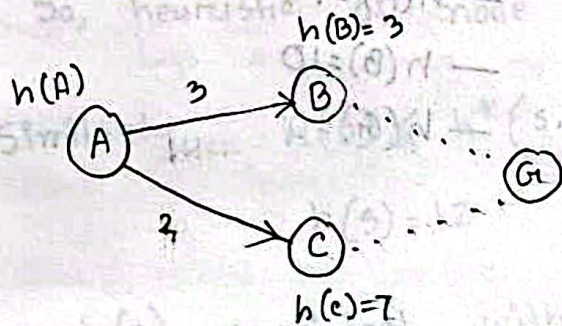
Q Which one is better? Admissible or inadmissible?

Q If you have two heuristic function, is it possible to choose which one is better?

↳ larger / dominating one.

Consistency

- heuristic can be admissible but not consistent.
- A consistent heuristic is by default admissible
- Admissibility relies on expanding a node more than once.
- If a node has a heuristic value not consistent, heuristic function not consistent. Then graph search version of A^* is no longer consistent.



consistent if:

$$h(A) \leq h^*(A,B) + h(B)$$

$$\leq 3 + 3$$

$$\leq 6 \quad \checkmark$$

$$h(A) \leq h^*(A,C) + h(C)$$

$$\leq 2 + 7$$

$$\leq 9$$

- A consistent heuristic is by default admissible.

→ Graph Search →

→ Tree Search →

$$0 = (A)N, \quad 5 > (B)N, \quad 0 = (A)N, \quad 4 > (C)N \quad \therefore$$

It is better to check consistency from the root node.

Informed search strategies are particularly useful for finding paths to the goal state.

For example, solving the 8-puzzle requires a series of consecutive actions to reach the solution; similarly, traveling from Arad to Bucharest involves a sequence of actions moving from one connected city to another.

Local search strategies focus solely on generating a goal state or a good state, regardless of the specific actions taken.

For example, in a simplified knapsack problem, the solution involves selecting a set of items that maximizes reward without exceeding the weight limit. The process does not concern itself with the order in which items are checked or selected to achieve the maximum reward.

→ It is especially useful in problems with very large search spaces.

It is better to start checking consistency from the goal state.

Local Search (L4)

- ⇒ Simple for optimization
- ⇒ The path to the goal is irrelevant
- ⇒ not the best solution but a decent one
- ⇒ These algorithms do not allow backtrack
- ⇒ Start the search with a randomly generated solution
 - ↳ In every iteration they do something to get the better version of the previous solution.

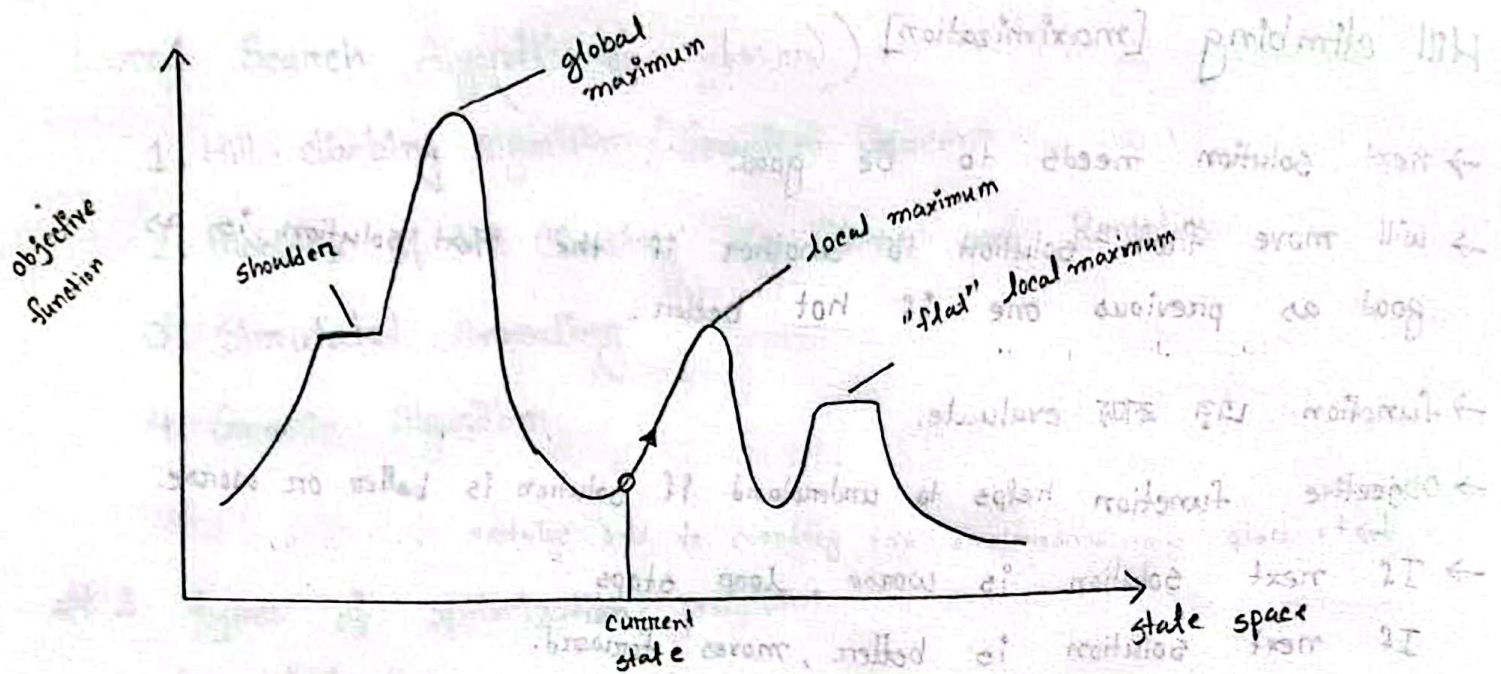
Need for Local Search

- Large or poorly understood search spaces, where exhaustive global search is impractical.
- Optimization tasks, focusing on improving a measure rather than finding a path.
- Dynamic problems, where solutions must adapt to changes without restarting the search.
- Local search also complements hybrid algorithms, combining it's strengths with other strategies for enhanced performance across various problems.

Local Search algorithms are not systematic - they might never explore a portion of the search space where a solution actually resides. However, they have two key advantages:

1) They use very little memory

2) They can often find reasonable solutions in large or infinite state spaces for which systematic algorithms are unsuitable.



Drawbacks:

- 1] Local Maxima
- 2] Plateaus $\rightarrow$ remains constant
- ?? 3] Ridges - (are sequences of local maxima that make it very difficult for greedy algorithm like hill climbing to navigate effectively)

To overcome this we use

- random restart (trial and error)
- Problem reformation (state change)
- Simulated Solution

☐ Simulated Annealing (Temperature)

→ Simulated Annealing (SA) is a probabilistic technique for approximating the global optimum of a given function.

→ The algorithm iterates by generating small modifications to the current solution, evaluating the cost changes, and probabilistically deciding whether to accept the new solution based on the current temperature.

1) increase temp high

2) can't keep same temp

3) decrease slowly

Key Parameters:

- Initial Temperature: High enough to allow exploration.

- Cooling Rate: Determines how quickly the temperature decreases.

- Stopping Temperature: Low enough to stop the process once the system is presumed to have stabilized.

Common Types of Mutation

- 1) Bit Flip Mutation (for binary encoding)
- 2) Random Resetting (for integer or real values)
- 3) Swap Mutation
- 4) Scramble Mutation
- 5) Uniform Mutation (for real-valued encoding)

Mutation Rate:

- Defines the probability with which a mutation will occur in an individual gene.
 - high M.R → diversity increase, disrupt highly fit solution
 - lower M.R → not enough diversity, leading premature convergence
- Optimal mutation Rate often depends on the specific problem and the characteristics of the population.

Diversity in Genetic Algorithm: initially more diverse → state space takes large space eventually less diverse → smaller steps are typical

- It is important for the initial population to be diversified. Otherwise similar type of chromosomes will crossover to and produce offsprings with little change in their fitness.
- ↳ This will lead to quick convergence of the algorithm and the chances of finding a solution with maximum fitness will be very low.

Advantages and Applications

Genetic algorithms are useful when:

- The search space is large, complex, or poorly understood.
- Traditional optimization and search techniques fail to find acceptable solutions.
- The problem is dynamic and changes over time, requiring adaptive solutions.

Genetic Algorithm
examples

Adversarial Search (L-5)

Two player zero-sum games

- **Deterministic**: We can determine the outcome of an action.
- **Two players**: Only two agents working against each other. Example: Player A and Player B.
- **Turn Taking**: Agents take turns alternatively. Example: After Agent A plays a move, Agent B will get to play their move and so on.
- **Perfect Information**: Fully observable environment, as in there are no parts of the game unknown to the agents.
- **Zero-Sum Games**: The total sum of points of the players in the game is zero. If one player wins (+1 points) that would mean the opponent player loses (-1 points). The total sum of the points in the game will be zero. When there is a draw, the point for both players is zero, meaning the sum will be zero.

☐ Making optimal decisions using Minimax Search

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$

not feasible

Minimax Algorithm $\rightarrow$ why is it impractical/??? $\Rightarrow$ branching factor 3, depth = 80

$\rightarrow$ Explores the game tree using a depth-first search. $\hookrightarrow$ almost 10^{123} states.

$\rightarrow$ If the tree's maximum depth is m and there are b legal moves at each point, the time complexity is $O(b^m)$ and the space complexity is $O(bm)$.

✖✖

$\rightarrow$ The exponential complexity makes minimax impractical for complex games

Algorithm

function MINIMAX-SEARCH (game, state) returns an action

player $\leftarrow$ game.TO-MOVE(state)

value, move $\leftarrow$ MAX-VALUE(game, state)

return move

function MAX-VALUE (game, state) returns a (utility, move) pair

if game.IS-TERMINAL (state) then return game.UTILITY (state, player), null

$v \leftarrow -\infty$

for each a in game.ACTIONS (state) do

$v_2, a_2 \leftarrow$ MIN-VALUE (game, game.RESULT (state, a))

if $v_2 > v$ then

$v, \text{move} \leftarrow v_2, a_2$

return v, move

function MIN-VALUE (game, state) returns a (utility, move) pair

if game.IS-TERMINAL (state) then return game.UTILITY (state, player), null

$v \leftarrow +\infty$

for each a in game.ACTIONS (state) do

$v_2, a_2 \leftarrow$ MAX-VALUE (game, game.RESULT (state, a))

if $v_2 < v$ then

$v, \text{move} \leftarrow v_2, a_2$

return v, move

Alpha Beta Pruning Algorithm

Max(s, α, β) {

if s is leaf node:

return Utility(s)

$V = -\infty$

for each child of s :

$V' = \text{Min}(s, \alpha, \beta)$

if $V' > V$:

$V = V'$

pruning $\rightarrow$ if $V' > \beta$:

return V

if $V' > \alpha$:

$\alpha = V'$

return V }

Min(s, α, β) {

if s is leaf node:

return Utility(s)

$V = +\infty$

for each child of s :

$V' = \text{Max}(s, \alpha, \beta)$

if $V' < V$:

$V = V'$

pruning $\rightarrow$ if $V \leq \alpha$:

return V

if $V' < \beta$:

$\beta = V'$

return V }

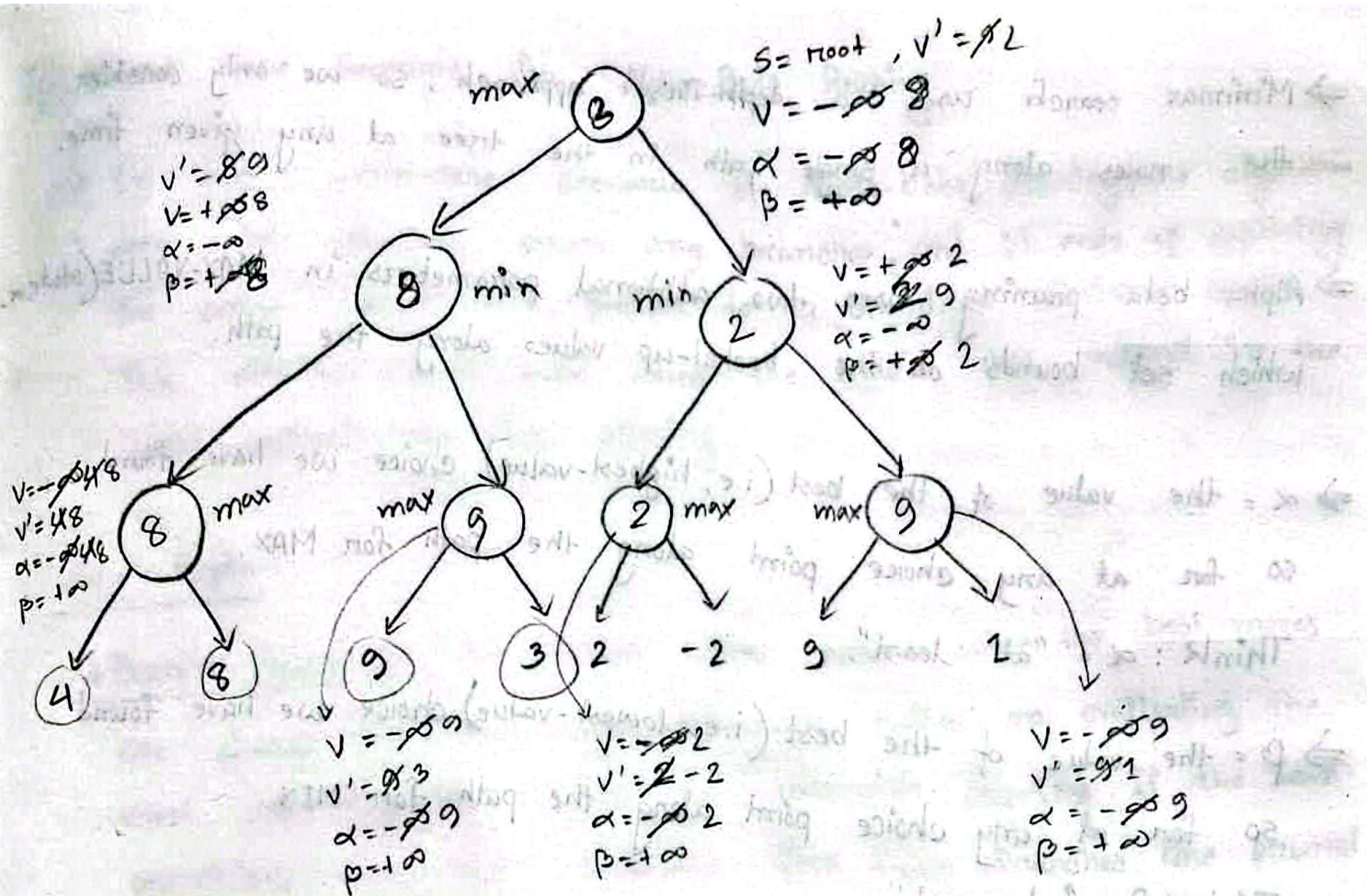
max($\text{root}, \alpha = -\infty, \beta = +\infty$)

$\alpha \rightarrow$ the best value that Max can guarantee so far.

$\beta \rightarrow$ the best value that Min can guarantee so far.

*if max finds a move better than β , Min will never allow it $\rightarrow$ stop searching that branch (prune).

*if min finds a move worse than α , Max will never allow it $\rightarrow$ stop searching that branch (prune).



and p is the maximum depth of the tree. $O(p^{min})$ is $O(p^{max})$ where p is the balancing factor.

→ Strategic planning provides a framework for decision-making in complex, changing environments. It involves setting long-term goals and determining the best way to achieve them.

★ ★ ★ Always follow these steps

1) Pop

* Priority Queue

2) Expand

3) Goal Test

* Frontier / Fringe → a data structure used to store the set of all possible states or nodes that are candidates for exploration next in a search process.

- Greedy Search is not optimal.
- Greedy search is incomplete without systematic checking of repeated states.

- In the worst case, the Time and space Complexity of Greedy Search are both $O(b^m)$

where b is the branching factor and m the maximum path length.

By applying Graph Search version, you can make

Greedy Search Complete.

Space

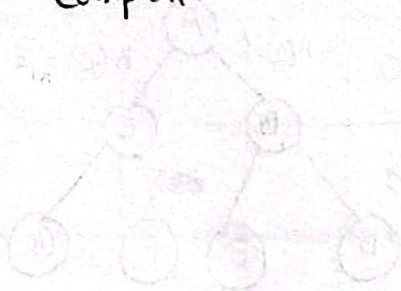
Optimal?

Yes

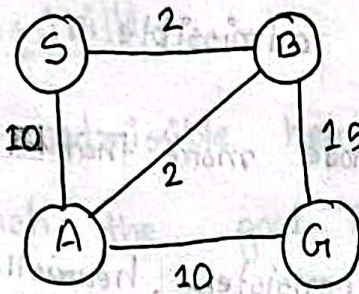
No

No

Yes



Example: Checking inconsistency



• Edges with costs:

- S to A = 2
- S to G = 3
- S to B = 1
- B to G = 10

Admissibility:

• Checking consistency for each node:

• Heuristic (h) Estimates to the Goal (G):

$$h(S) = 12$$

$$h(A) = 7$$

$$h(B) = 10$$

$$h(G) = 0$$

- Optimal path from Node S is:

$$S - B - A - G = 2 + 2 + 10 = 14$$

- Optimal path from Node B is:

$$B - A - G = 2 + 10 = 12$$

- Optimal path from Node A is:

$$A - G = 10$$

- Optimal path from Node G is:

$$G = 0$$

$$\therefore h(S) < 14, h(A) < 10, h(B) < 12, h(G) = 0$$

consistency :

It is better to start checking consistency from the Goal node.

If you have two heuristic function, is it possible to choose which one is better?

→ larger / dominating one.

Local Search Algorithms:

1. Hill-climbing Algorithm / Gradient Descent
2. Problems of Hill Climbing Algorithms and Remedies
3. Simulated Annealing
4. Genetic Algorithm

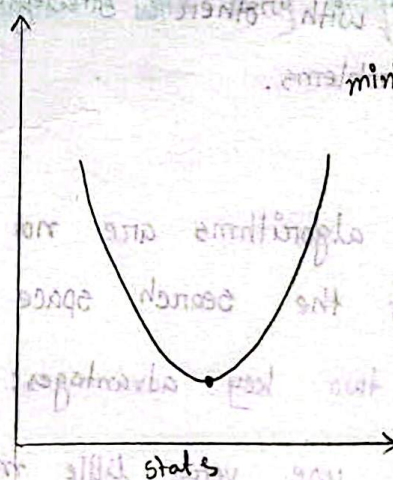
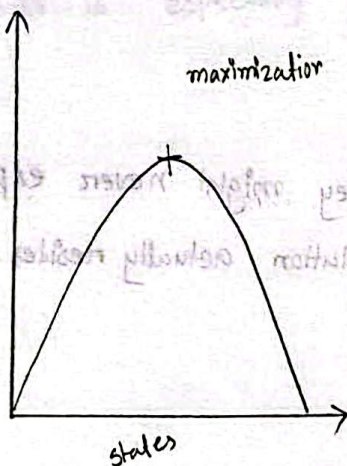
2 types of optimization problem

↳ maximization.

↳ minimization

Traveling Salesmen Problem (TSP) [minimization]

→ Salesmen start from random city, visit every single city in least possible distance but visit once then come to initial state.



multiply with -1 to turn into maximization/minimization.

or divide by -1

Genetic Algorithm ***

Genetic algorithms are a class of optimization algorithms inspired by the principles of natural selection and genetics. These algorithms are used to solve search and optimization problems and are particularly effective for complex issues that are difficult to solve using traditional methods.

Algorithm:

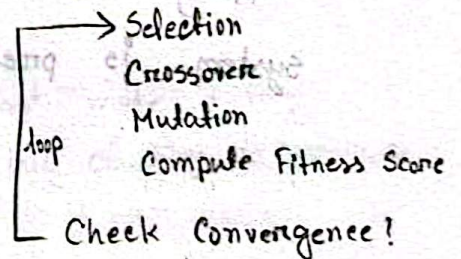
- 1] Input : Population size, fitness function, mutation rate, crossover rate, maximum generations
- 2] Output: Best solution found
- 3] begin
- 4] Initialize population with random candidates
- 5] Evaluate the fitness of each candidate
- 6] While termination condition not met do
- 7] Select parents from the current population
- 8] Perform crossover on parents to create new offspring
- 9] Perform mutation on offspring
- 10] Evaluate the fitness of new offspring
- 11] Select individuals for the next generation
- 12] best solution in new generation > best solution so far
- 13] Update best solution found
- 14] end While
- 15] return best solution found
- 16] end

Pseudocode

Start

Initial Population (Random)

Compute fitness score



↓ Yes
Stop

Maximizer and Minimizer

Max:

- ⇒ MAX aims to maximize its own game points.
- ⇒ In a zero-sum game, the total payoff for all players is constant, meaning one player's gain is exactly equal to another's loss.
- ⇒ MAX seeks to achieve the highest possible outcome for itself, knowing that this will occur at the expense of the other player.
- ⇒ MAX evaluates all possible moves by considering what its opponent might choose, countering their moves. It chooses the option that leads to the highest evaluated payoff under optimal play.

Min:

- ⇒ Min aims to win by minimizing MAX's points. While choosing its move, MIN selects the move that will reduce the good options for MAX, limiting MAX's game points and forcing it towards an outcome where MAX's game points are minimal.
- This strategy minimizes Min's loss as well.

- ⇒ For example, in a simple game like tic-tac-toe, MAX aims to align three of its marks in a row, thereby maximizing its chance of winning and ensuring the other player's loss. Conversely, MIN attempts to block MAX's efforts to align three marks, while also seeking opportunities to create a threat that MAX must respond to, thereby steering the game towards a draw or a win for MIN.

Python

```
def minimax_search(game, state):
```

```
    player = game.to_move(state)
```

```
    value, move = max_value(game, state)
```

```
    return move
```

```
def max_value(game, state):
```

```
    if game.is_terminal(state):
```

```
        return game.utility(state), None
```

```
    v = float('-inf')
```

```
    best_move = None
```

```
    for action in game.action(state):
```

```
        v2, a2 = min_value(game, game.result(state, action))
```

```
        if v2 > v:
```

```
            v = v2
```

```
            best_move = action
```

```
    return v, best_move
```

```
def min_value(game, state):
```

```
    if game.is_terminal(state):
```

```
        return game.utility(state), None
```

```
    v = float('inf')
```

```
    best_move = None
```

```
    for action in game.action(state):
```

```
        v2, a2 = max_value(game, game.result(state, action))
```

```
        if v2 < v:
```

```
            v = v2
```

```
            best_move = action
```

```
    return v, best_move
```


A* Search

- 1) Priority Queue
- 2) Path
- 3) Cost

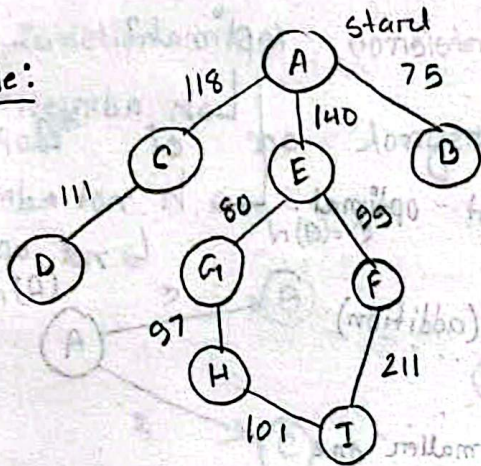
→ evaluation function, $f(n) = g(n) + h(n)$

$g(n)$ is the path cost from initial state to node n,

$h(n)$ is the estimated cost of the shortest path from n to a goal state

A guarantees optimality under certain condition.

Example:



State	$h(n)$
A	366
B	374
C	329
D	244
E	293
F	178
G	193
H	98
I	0

0 → indicates goal state

A366 [B449 C447 E393]

E393 [B449 C447 G413 F417]

G413 [B449 C447 F417 H415]

H415 [B449 C447 F417 I418]

F417 [B449 C447 I418 I450]

I418 [B449 C447]

→ check / goal test ✓

∴ path : A → E → G → H → I

Total Cost = $140 + 80 + 97 + 101$
= 418

Parent	X	A	A	A	E	E	G	H
--------	---	---	---	---	---	---	---	---

	A	B	C	D	E	F	G	H	I
--	---	---	---	---	---	---	---	---	---

$$f(A) = g(A) + h(A) = 0 + 366 = 366$$

$$f(B) = g(B) + h(B) = 75 + 374 = 449$$

$$f(C) = g(C) + h(C) = 118 + 329 = 447$$

$$f(E) = g(E) + h(E) = 140 + 293 = 393$$

$$f(G) = g(G) + h(G) = 140 + 80 + 193 = 413$$

$$f(F) = g(F) + h(F) = 140 + 99 + 178 = 417$$

$$f(H) = g(H) + h(H) = 140 + 80 + 97 + 98 = 415$$

$$f(I) = g(I) + h(I) = 140 + 80 + 97 + 101 = 418$$

$$f(I) = g(I) + h(I) = 140 + 99 + 211 = 450$$

⇒ Minimax search uses a depth-first approach, so we only consider the nodes along a single path in the tree at any given time.

⇒ Alpha beta pruning uses two additional parameters in MAX-VALUE^(state) which set bounds on the backed-up values along the path.

⇒ α = the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX.

Think : α = "at least".

⇒ β = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN.

Think : β = "at most".

⇒ In the best case, alpha-beta pruning can reduce the time complexity from $O(b^m)$ to $O(b^{m/2})$ where b is the branching factor and m is the maximum depth of the tree.

⇒ Alpha-beta pruning enables more effective decision-making in complex games, allowing for deeper searches and better strategic planning within practical time limits.

We see $h(G) = 0$ is admissible and consistent.

$$\therefore h(A) \leq h^*(A, G) + h(G) = 10 + 0 = 10$$

$$h(A) = 7$$

So, node A is consistent with node G.

$$\therefore h(B) \leq h^*(B, A) + h(A) = 2 + 7 = 9$$

$$h(B) = 10$$

So, heuristic of node B with node A is inconsistent.

$$\text{Similarly, } h(s) \leq h^*(s, B) + h(B) = 2 + 10 = 12$$

$$h(s) = 12$$

$\therefore h(s)$ consistent with node B.

completeness

$\hookrightarrow$ Graph Search $\rightarrow h$ consistent $\rightarrow$ complete

$\hookrightarrow$ Tree Search $\rightarrow h$ admissible $\rightarrow$ complete.

Improving alpha-beta pruning

In complex games with large search spaces, to improve the efficiency and effectiveness of the alpha-beta pruning these following techniques can be used.

- Move-Ordering: Move-ordering prioritizes evaluating the most promising moves first, increasing the chances of effective pruning in alpha-beta search. This is usually achieved by sorting moves based on heuristic evaluations or previous search results, aiming to maximize the number of branches pruned.
- Killer Moves: Killer moves are specific moves that have previously caused significant pruning in similar positions and are tried early in the search to maximize pruning opportunities. These moves are stored and reused, assuming that moves which were effective in the past are likely to be effective again.
- Iterative Deepening: Iterative deepening involves repeatedly running depth limited searches with increasing depth limits, combining the benefits of depth-first search (using less memory) and breadth-first search (finding the optimal solution). It provides a way to use the best move from shallow searches to improve move-ordering in deeper searches.
- Heuristic Alpha-Beta: Heuristic alpha-beta uses evaluation functions to estimate the value of non-terminal nodes, guiding the search and pruning process more effectively based on heuristics. These evaluation functions consider factors like material balance, positional advantages and other domain-specific knowledge to approximate the true minimax value of a position.

Hill climbing [maximization] (Greedy approach)

- next solution needs to be good.
- will move from solution to another if the next solution is as good as previous one if not better.
- function is evaluated.
- Objective function helps to understand if solution is better or worse.
 - ↳ to help you understand the goodness of the solution.
- If next solution is worse, loop stops
- If next solution is better, moves forward.

No back tracking

Algorithm:

1. Input: initial-state, objective-function()
2. Output: state that is a local maximum
3. begin
4. current-state $\leftarrow$ initial-state
5. loop do
6. neighbor $\leftarrow$ a state that is a neighbor of current-state
7. If objective-function(neighbor) $\leq$ objective-function(current-state)
8. Return current-state
9. current-state $\leftarrow$ neighbor

Gradient descent algorithm:

When the goal of the hill-climbing algorithm is to find the state with the minimum value rather than maximizing the objective, it is known as gradient descent.

Explanation of the Pseudocode

Initialize Population:

A genetic algorithm begins with a population of randomly generated individuals. Each individual, or chromosome, represents a possible solution to the problem. Depending on the problem the chromosomes are encoded.

Evaluate Fitness:

Each solution in the population is assessed using the fitness function to determine how well it solves the problem. Depending on the problem the fitness function will be measured.

Selection:

This step involves choosing the fitter individuals to reproduce.

Truncation Selection: Only the top-performing fraction of the population is selected to reproduce.

Crossover: (Recombination)

Pairs of individuals are crossed over at a randomly chosen point to produce offspring. The crossover rate determines how often crossover will occur. [Single-point crossover and double-point crossover]

Mutation:

With a certain probability (mutation rate), mutations are introduced to the offspring to maintain genetic diversity within the population.

Purpose:

↳ Introduce Variation

↳ Prevent Local Optima

↳ Explore New Areas

⇒ MAX and MIN alternatively choose moves, each striving to reach their respective goals.

⇒ The minimax algorithm provides a way to determine the best possible move for a player by minimizing the possible loss for a worst case scenario.

*state can be reached by multiple paths.

① ⇒ state space graph
↳ vertices = states
↳ edges = moves

The following elements formally define the game:

① • S_0 : The initial state, which specifies how the game is set up at the start.

• TO-MOVE(s): The player whose turn it is to move in state s .

① • ACTIONS(s): The set of legal moves in state s .

① • RESULT(s, a): The transition model, specifying the state that results from executing action a in the state s .

• IS-TERMINAL: A test to determine if state s is terminal, returning true if the game has concluded and false otherwise. States where the game has ended are called

The numbers on each terminal states. Example, Win, lose, Draw.

↳ leaf node represents the utility value.

• UTILITY: A utility function (also known as an objective function or payoff function), which provides the final numerical value to player P when the game concludes in a terminal state s . For instance, in chess, the result can be a win, loss, or draw, with values 1, 0, or $\frac{1}{2}$ respectively. In some games, the payoff range can be broader, such as in backgammon, where it ranges from 0 to 192.

Alpha-Beta Pruning

→ Alpha-beta pruning is an optimization technique for the minimax algorithm, which is used in decision-making process two-player games.

→ The primary goal of alpha-beta pruning is to reduce the number of nodes evaluated in the game tree, thereby improving the efficiency of the minimax algorithm.

→ Minimax algorithm is an exhaustive search that can become computationally expensive, especially for complex games with large search spaces.

→ Alpha beta addresses this issue by eliminating branches in the game tree that do not influence the final decision.

❏ Worst Case Scenario for Alpha-Beta Pruning

⇒ In the worst-case scenario of alpha-beta pruning, the algorithm does not effectively prune any branches, and it ends up exploring the entire game tree, similar to a standard minimax search.

This situation can arise when the nodes are ordered in the least optimal way for pruning.

Key Points :

- Branch Ordering: The worst case occurs when the best moves are always considered last. Alpha-beta relies on evaluating the most promising moves first to maximize pruning. If the least promising moves are evaluated first, fewer branches are pruned.

- No Pruning: When the algorithm does not prune any branches, it evaluates every possible move at each depth level, leading to the maximum number of nodes being explored.

- Time Complexity: In the worst case, the time complexity of alpha-beta pruning is the same as that of the minimax algorithm without pruning, which is $O(b^m)$. Here, b is the branching factor (the number of legal moves at each point), and m is the maximum depth of the tree.